1 Shizuku V3 カーネル解説: method call と svc の軌跡

状態: **解説記事 (一次資料に基づく技術解説)**。2026-10-07。
読者と目的: Shizuku の作者本人が、V3 カーネル設計書 (`docs/design/kernel_v3.typ`) を読み解くための手引き。
特に **method call と svc の実行軌跡を、V2 と V3 で、スタック (PSP/MSP、例外フレーム、callee-saved、call frame / ledger) とレジスタ (r0–r3、LR/EXC_RETURN、CONTROL、PSPLIM、MPU) の変化を 1 ステップずつ追って** 比較・解説する。

本書で用いるラベル:

- **[コード]**: 現行リポジトリのソースコード (`modules/pico_sdk_support/rp2350/`, `source/kernel/`, `internal_headers/`) に実在する実装。
- **[実測]**: `cost3_report.md`、`cost2_trace.log` 等で実機 (RP2350 150MHz) 採取された確定データ。
- **[設計]**: `docs/design/kernel_v3.typ` で策定された V3 の確定決定事項。
- **[未確定]** / **[棚上げ]**: 設計書内で未確定または棚上げと明記されている事項。

—

1.1 前提: Cortex-M33 の例外動作と Shizuku のアーキテクチャ語彙

1.1.1 Cortex-M33 (ARMv8-M Mainline) の例外入口・復帰とレジスタ規約

Shizuku のカーネル・オブジェクト境界は、ARMv8-M の例外機構に全面的に依存している。まずハードウェアの基本動作を整理する。

ハードウェア要素	動作・仕様・Shizuku における役割
ハードウェア Stacking	Thread mode で `svc 0` を発行すると、ハードウェアが自動的に 8 語 (32B: `r0`, `r1`, `r2`, `r3`, `r12`, `LR`, `PC`, `xPSR`) をアクティブなスタック (通常は PSP) に push する。
8B スタック整列パディング	SVC 発行直前に SP が 8B 整列でない場合、ハードウェアは 4B パディングを挿入して 8B 整列にし、`xPSR` の bit 9 (SPREALIGN) を 1 に立てる (`armv8m.hpp:98–108`)。例外復帰時に自動で +4 補正される。
Lazy FP Stacking	FPU が活性化している場合 (`FPCCR.LSPACT=1` または `CONTROL.FPCA=1`)、ハードウェアは追加で 18 語 (72B: `s0–s15`, `FPSCR`, 予約語) の領域を予約し、計 26 語 (104B) の拡張フレームを形成する (`armv8m.hpp:86, 96`)。実体化は FP 命令実行時まで遅延される。
`EXC_RETURN`	例外突入時、`LR` レジスタには特殊な戻りトークンが装填される。通常は `0xFFFFFFFD` (bit 4=1: 標準フレーム、bit 3=1: Thread mode 復帰、bit 2=1: PSP スタック復帰)。Handler mode から `bx lr` することでハードウェア unstacking が走り例外復帰する (`armv8m_ctx.S:47–58, 115–116`)。
MSP と PSP	MSP (Main Stack Pointer) は Handler mode (割り込み・例外ハンドラ) で使用される。PSP (Process Stack Pointer) は Thread mode (ユーザーオブジェクトおよび Thread カーネル) で使用される。
`CONTROL` レジスタ	bit 0 (`nPRIV`: 0=特権, 1=非特権)、bit 1 (`SPSEL`: 0=MSP, 1=PSP)、bit 2 (`FPCA`: FPU 活性)。Shizuku は Thread mode 実行中常に `SPSEL=1` (PSP) を維持する (`armv8m_ctx.S:91–102`)。
`PSPLIM` レジスタ	ハードウェアスタック下限。PSP が `PSPLIM` を下回ると即座に UsageFault (HardFault へエスカレーション) が発火する (`armv8m.hpp:76`, `dispatch.cpp:84–109`)。
PMSAv8 MPU	RP2350 の保護機構。RBAR (Base) と RLAR (Limit) で 32B 整列の保護区画を指定する。Shizuku では Region 2 を動的な `GRANT_REGION` 窓として使用 (`arch_glue.cpp:56–76`)。

1.1.2 Shizuku のアーキテクチャ語彙

設計書とコードを読み解く上で必須となる Shizuku 固有の概念を定義する。

- **カーネル (kernel)**: `source/kernel/dispatch.cpp` に実装される最下層。機構 (mechanism) のみを提供し、コンテキスト切替、フレームの push/pop、MPU 窓の反映、スケジューリングの primitive を担う。V2 ではオブジェクトの概念を知らない (D1)。
- **カーネルオブジェクト (kernel object / kobj)**: `source/kernel_object/handler.cpp` に実装される方針 (policy) の提供者。システムリソース (スレッド、オブジェクト、ストリーム、メモリ) を管理する。V2 では Thread mode 上の独立したハンドラ (`object_kind::HANDLER` / `KERNEL_OBJECT`) として実行される。
- **アクター (actor)**: 保護ドメインと実行権限の論理単位。V2 では各オブジェクトが固有の ID を持つ。

- **台帳 (call ledger)**: 特権メモリ (`g_bookkeeping_storage`) 上に確保された呼び出し履歴管理構造体 (`templates/thread.hpp:41–48`)。呼び出し元/先オブジェクト ID、種別 (`kind`)、経路 (`via`) 等を記録し、非特権オブジェクトからは改ざんできない。
- **コールフレーム (call frame)**: スレッドの PSP スタック上に積まれるフレーム。呼び出し元のコンテキスト退避域 (`call_frame_header`) と、ハードウェア例外フレームの作業コピーから構成される (`dispatch.cpp:69–144`)。
- **コンテキスト再利用 (context reuse)**: 別アクターを呼び出す際、新しいスレッドを割り当てる (`SPAWN`) のではなく、同一スレッドの同一 PSP スタック上でコンテキストを退避・積み替えて実行を遷移させる手法 (`dispatch.cpp:69–159`)。
- **オブジェクトランド (objectland)**: 非特権または特権で動作するオブジェクト群の実行環境。親ハンドラ (`parent_handler`) が未登録の raw SVC を捕捉・中継する階層構造を持つ (`dispatch.cpp:309–358`)。

—

1.2 V2 の SVC 1 回 (GET_CURRENT_OBJECT) の詳細追跡

1.2.1 2 回の境界越えの全貌

V2 において、最も単純な同期システムコールである `GET_CURRENT_OBJECT` (API ID=5) を発行した場合の実行軌跡を追う。この呼び出しは、実測で **SVC 2 回、例外復帰 2 回、push 1 回、pop 1 回、MPU 更新 2 回** を消費する (`cost3_report.md:57`)。

なぜ 1 回の API 呼び出しに 2 回の SVC が必要なのか？ その根本原因は、**「カーネル (Handler mode)」と「カーネルオブジェクト (Thread mode)」が分離しており、カーネルオブジェクトが通常の C 関数として return したあと、カーネルの巻き戻しプリミティブ (`primitive::RETURN`) を呼ぶためにもう一度 SVC を発行する構造** にある。

境界	実行モード / 主体	処理内容	コード位置

SVC 1	Thread (非特権) → Handler	Caller が `GET_CURRENT_OBJECT` を発行。ハードウェア stacking 後、`CTX_SAVE`。親 kobj へのトランポリン作成 (`call_frame_push`)。	`armv8m.hpp:406–419`
			`armv8m_ctx.S:126–130`
			`dispatch.cpp:309–358`

復帰 1	Handler → Thread (特権)	`CTX_RESTORE` 後、`bx lr` で例外復帰。復帰先は kobj の `handler_entry_point`。	`armv8m_ctx.S:65–117` `handler.cpp:122–128`
処理	Thread (特権)	kobj が `handle()` を実行。`current_object()` を取得して通常の C return。戻り先は `return_stub`。	`handler.cpp:1608–1610` `handler.cpp:1696`
SVC 2	Thread (特権) → Handler	`return_stub` が `primitive::RETURN` (count=1) を発行。`CTX_SAVE` 後、`call_frame_pop` で caller の元コンテキストを復元。	`armv8m_ctx.S:157–165` `dispatch.cpp:249–277`
復帰 2	Handler → Thread (非特権)	`CTX_RESTORE` 後、`bx lr` で例外復帰。復帰先は caller の SVC 直後。	`armv8m_ctx.S:65–117` `armv8m.hpp:418`

1.2.2 各時点のスタック断面とレジスタ推移

この 2 回の境界越えにおける PSP スタック、MSP スタック、主要レジスタの変化をステップごとに追跡する。

1.2.2.1 [時点 1] Caller が SVC を発行する直前

- **モード**: Thread mode、非特権 (`CONTROL.nPRIV=1`, `CONTROL.SPSEL=1`)
- **レジスタ**:
  - `r0`: 5 (`object_api::GET_CURRENT_OBJECT`)
  - `r1–r3`, `r12`: 0 (未使用引数)
  - `PSP`: P_0 (Caller のスタックトップ)
  - `MSP`: M_0 (カーネルの予約 MSP)
  - `PSPLIM`: Caller のスタック下限 L_{caller}

1.2.2.2 [時点 2] SVC 1 発行直後 (ハードウェア Stacking 完了、Handler mode 突入)

- **ハードウェア動作**:
  - PSP 上に 8 語 (32B) の HW Frame A を自動 push。
  - $P_1 = P_0 - 32$ (整列パディングがある場合は −36)。
  - `LR`: `0xFFFFFFFD` (`EXC_RETURN`: Return to Thread using PSP, standard frame)。
  - モード: Handler mode、特権。

アドレス	スタック構造	内容
P_0 (高位)	Caller ローカル領域	呼び出し元のローカル変数・C スタックフレーム
P_1 .. P_0 (低位)	**HW Frame A (32B)**	r0(=5), r1(=0), r2(=0), r3(=0), r12(=0), LR(=Caller LR), PC(=Caller SVC の次の命令), xPSR

1.2.2.3 [時点 3] CTX_SAVE 完了直後 (armv8m_ctx.S:32–60)

- push {lr} により MSP に EXC_RETURN が退避される ($M_1 = M_0 - 4$)。
- shizuku_current_context を呼び出し、現在スレッドの context_t (120B) を取得。
- r4–r11、PSP (P_1)、EXC_RETURN が Caller の context_t に退避される。

1.2.2.4 [時点 4] shizuku_svc_dispatch による call_frame_push 完了直後 (dispatch.cpp:69–143, 309–358)

- Caller は非 KERNEL_OBJECT であるため、親 kobj (ID 0) へディスパッチされる。
- スタックの幾何計算:
  - caller_stack = $P_1 + 32 = P_0$
  - total = sizeof(call_frame_header) + 32 = 120 + 32 = 152B (8B 整列済み)
  - snapshot = $P_0 - 152$ (ここに call_frame_header を配置)
  - callee_frame = snapshot - 32 = $P_0 - 184$
- header->saved = *context: Caller の 120B context_t (元の P_1 を含む) をヘッダへ完全コピー。
- memcpy(callee_frame, context->sp, 32): 元の HW Frame A を callee_frame へ複製。
- フレーム書き換え (ARCH::set_entry, ARCH::set_args, ARCH::set_priv):
  - callee_frame->PC = kobj.parent_handler_entry (handler_entry_point)
  - callee_frame->LR = ARCH::return_stub() (shizuku_armv8m_return_stub)
  - callee_frame->r0 = 5 (GET_CURRENT_OBJECT)
  - context->control: nPRIV を 0 (特権) に設定
  - context->sp = callee_frame ($P_0 - 184$)
- thread.call_stack.top = snapshot、depth = 1、ledger[0] に記録。

アドレス	スタック構造	内容
P_0 (高位)	Caller ローカル領域	呼び出し元のローカル変数
$P_0 - 32$	元の HW Frame A (32B)	保存されたまま温存される (変更しない)
$P_0 - 152$ .. $P_0 - 32$	**call_frame_header (120B)**	prev(=0), total_bytes(=152), frame_bytes(=32), saved (**Caller の 120B context_t**)
$P_0 - 184$ .. $P_0 - 152$	**callee_frame (32B)**	r0(=5), r1..r3(=0), r12(=0), LR(=**return_stub**), PC(=**handler_entry_point**), xPSR

1.2.2.5 [時点 5] CTX_RESTORE 1 と例外復帰 1 直後 (armv8m_ctx.S:65–117)

- context_t から callee_frame を PSP として復元。CONTROL は特権 (nPRIV=0)。
- bx lr によりハードウェアが callee_frame (32B) を unstack !
- **復帰後の状態**:
  - モード: Thread mode、**特権** (nPRIV=0, SPSEL=1)
  - PSP: $P_0 - 152$ (snapshot の位置)
  - PC: handler_entry_point
  - LR: shizuku_armv8m_return_stub
  - r0: 5 (GET_CURRENT_OBJECT)

1.2.2.6 [時点 6] kobj のハンドラ実行と return (handler.cpp:122–128, 1608–1610, 1696)

- handler_entry_point(5, 0, 0, 0) が kernel_object_instance.handle(5, 0, 0, 0) を呼び出す。
- case object_api::GET_CURRENT_OBJECT: により value = current_object(...) を取得。
- return value; (普通の C return)。
- handler_entry_point のエピローグで pop {pc} または bx lr。LR に仕込まれていた shizuku_armv8m_return_stub へジャンプする !

1.2.2.7 [時点 7] return_stub による SVC 2 発行 (armv8m_ctx.S:157–165)

- r0 = 2 (primitive::RETURN)
- r1 = 1 (落とす段数: count=1)
- r2 = value (戻り値)
- r3 = 0 (error=0)
- r12 = 0

- `svc 0` を発行！
- ハードウェアが PSP ($P_0 - 152$) に HW Frame B (32B) を push。$P_2 = P_0 - 184$。
- Handler mode へ突入 (`LR = 0xFFFFFFFD`)。

1.2.2.8 [時点 8] SVC 2 の `shizuku_svc_dispatch` による `call_frame_pop` (`dispatch.cpp:218–277`)

- 発行元スレッドは `KERNEL_OBJECT` 本人であるため、`primitive::RETURN` の分岐へ入る。
- `call_frame_pop`:
  - `header = (call_frame_header *)thread.call_stack.top` ($P_0 - 152$)
  - `*context = header->saved`: **Caller の 120B `context_t` を完全復元！**
  - この復元により、`context->sp` は元の HW Frame A ($P_1 = P_0 - 32$) を再び指す！
  - `thread.call_stack.top = header->prev` (=0)、`depth` = 0。
- 結果書き込み (`ARCH::set_result(*frame, error, value)`):
  - 復元された HW Frame A の `r0` に 0 (`kernel_error::OK`)、`r1` に `value` を書き込む。

1.2.2.9 [時点 9] `CTX_RESTORE 1` と例外復帰 2 直後

- `context_t` から Caller の元のレジスタ (`r4–r11`, `CONTROL.nPRIV=1` (非特権), `PSPLIM`) を復元。
- `bx lr` によりハードウェアが HW Frame A (32B) を unstack！
- **復帰後の状態**:
  - モード: Thread mode、**非特権** (`nPRIV=1`, `SPSEL=1`)
  - `PSP`: P_0 (Caller のスタックトップに完全復帰)
  - `PC`: Caller の SVC 0 の次の命令
  - `r0`: 0 (成功)
  - `r1`: `value` (現在のオブジェクト ID)

1.2.3 発生するオーバーヘッドと実測値 (1985 cyc の正体)

V2 の `GET_CURRENT_OBJECT` の実測値は **untraced net min 1985 cyc** (空枠控除前 raw min 2000 cyc) である (`kernel_v2_record.typ:20–25`, `cost3_report.md:81–83`)。 この 1985 cyc の内訳は、以下の通りである:

1. 2 回の境界越え (各境界で `CTX_SAVE` 約 131 cyc、dispatch 約 66 cyc、MPU wrapper 約 129 cyc、`CTX_RESTORE` 約 120 cyc、合計約 446 traced cyc × 2 = 約 892 cyc)。
2. 1 回の `call_frame_push` による 120B `context_t` 複写、例外フレーム複写、幾何検査 (約 432 traced cyc)。
3. Thread mode での kobj トランポリン実行 (`handler_begin`, `handle`, `current_object`, `return_stub`)。
4. 1 回の `call_frame_pop` による 120B `context_t` 読み戻し。

本来ならレジスタ 1 本を読んで返すだけの操作に、**特権 Thread モードへのコンテキスト積み替えと復帰を往復させたこと** が、2000 サイクル近くを費やす主因であった。

—

1.3 V2 の Method Call 1 段 (CALL_METHOD) の詳細追跡

1.3.1 5 回の SVC の大局観

V2 において、あるオブジェクトが別のオブジェクトのメソッドを呼ぶ `CALL_METHOD` (API ID=4) は、成功時に **SVC 5 回、例外復帰 5 回、push 3 回、pop 3 frame** を発生させる (`cost3_report.md:45–57`)。

登場する実行主体:

1. **Caller**: メソッドを呼び出す側 (Thread mode, 非特権)
2. **カーネル**: 機構を提供する Handler mode
3. **親 kobj**: 方針・台帳を管理する特権ハンドラ (Thread mode, 特権)
4. **Leaf**: 呼び出される末端メソッド (Thread mode, 特権または非特権)

SVC	発行元 (Thread)	カーネルの処理 (Handler)	例外復帰先 (Thread)	フレーム操作
1	Caller: `CALL_METHOD`	`dispatch.cpp:309–358` 親 kobj へ転送	kobj `call_method`	**push A** (dispatch frame)
2	kobj: `primitive::CALL`	`dispatch.cpp:231–248 do_call` 実行	Leaf 本体 (`LR=return_stub`)	**push B** (call frame)
3	Leaf 戻り: `return_stub`	`dispatch.cpp:309–358` 親 kobj へ転送	kobj `exit_method`	**push C** (dispatch frame)

SVC	発行元 (Thread)	カーネルの処理 (Handler)	例外復帰先 (Thread)	フレーム操作
4	kobj: `primitive::RETURN` (count=2)	`dispatch.cpp:249–277` C と B を pop	kobj `call_method` 継続部	**pop C + B** (2 frames)
5	kobj 戻り: `return_stub`	`dispatch.cpp:249–277` A を pop	元 Caller	**pop A** (1 frame)

1.3.2 5 つの境界のステップバイステップ追跡

1.3.2.1 [境界 1] Caller → 親 kobj (push A)

- **Caller** が `CALL_METHOD` (a0=4, a1=target_object, a2=method, a3=arg) を SVC 発行。
- **カーネル** は Caller が非 KERNEL_OBJECT であるため、親 kobj へ転送。
- `call_frame_push` により、Caller のスタック上に **Frame A** (`call_frame_header` 120B + `callee_frame` 32B) を push。
- 例外復帰 1: 復帰先は kobj の `handler_entry_point` → `kernel_object_instance.handle()` → `call_method()` (`handler.cpp:532–581`)。

1.3.2.2 [境界 2] 親 kobj → Leaf (push B)

- **kobj** の `call_method()` が対象オブジェクトの存在、メソッド未登録の有無、深さ制限を検査。
- `call_request` 構造体を構築し、`ARCH::syscall(primitive::CALL, &request)` を発行 (`handler.cpp:570–571`)。これが **SVC 2**。
- **カーネル** は発行元が KERNEL_OBJECT 本人であることを確認し、`do_call` (`dispatch.cpp:164–193`) を実行。
- `call_frame_push` により、スタック上に **Frame B** (Leaf 呼び出し用フレーム) を push。
- `ARCH::set_entry` で PC を Leaf の関数先頭、`LR` を `shizuku_armv8m_return_stub` に設定。
- 例外復帰 2: 復帰先は Leaf 関数本体 (`cost_leaf`)。

1.3.2.3 [境界 3] Leaf 戻り → 親 kobj EXIT_METHOD (push C)

- **Leaf** 関数が処理を終え、通常の C return を実行。
- `LR` に設定されていた `shizuku_armv8m_return_stub` (`armv8m_ctx.S:157–165`) に落ちる。
- `return_stub` は `movs r0, #2` (`primitive::RETURN`)、`movs r1, #1` をセットして `svc 0` を発行。これが **SVC 3**。
- **カーネル** の判定: Leaf は非 KERNEL_OBJECT である！
  - したがって `primitive::RETURN` としては解釈されない。
  - 親ハンドラへの転送経路 (`dispatch.cpp:309–358`) へ送られる。
  - しかも `r0=2` は `object_api::EXIT_METHOD` の番号と完全に一致する (`object_api.hpp:23–29`)！
- `call_frame_push` により、スタック上に **Frame C** (kobj の `exit_method` 呼び出し用フレーム) を push。
- 例外復帰 3: 復帰先は kobj の `exit_method()` (`handler.cpp:584–618`)。

1.3.2.4 [境界 4] 親 kobj EXIT_METHOD → kobj call_method 継続 (pop C + B)

- **kobj** の `exit_method()` が呼ばれる。
- kobj は自身の台帳から、落とすべき段数 `count = 2 * pops = 2` (Leaf の Frame B と、自分自身の Frame C の計 2 枚) を算出 (`handler.cpp:603`)。
- `ARCH::syscall(primitive::RETURN, count=2, value, error, depth)` を発行 (`handler.cpp:613`)。これが **SVC 4**。
- **カーネル** は発行元が KERNEL_OBJECT 本人であることを確認し、`primitive::RETURN` を処理。
- `count=2` であるため、`call_frame_pop` を 2 回連続で実行！ (**Frame C と Frame B が pop される**)。
- 復元されたコンテキストは、kobj の `call_method()` が SVC 2 を撃った直後の状態！
- 例外復帰 4: 復帰先は kobj の `call_method()` 内、`ARCH::syscall` の直後 (`handler.cpp:572`)。
- `call_method()` は Leaf の戻り値を受け取って `return result.value` を実行。`handle()` が return し、`handler_entry_point` が return。
- 戻り先アドレス `LR` は、境界 1 で仕込まれていた `shizuku_armv8m_return_stub` を指している！

1.3.2.5 [境界 5] 親 kobj 戻り → 元 Caller (pop A)

- `return_stub` が `movs r0, #2` (`primitive::RETURN`)、`movs r1, #1` をセットして `svc 0` を発行。これが **SVC 5**。
- **カーネル** の判定: 今度は発行元が KERNEL_OBJECT 本人である！
- `count=1` であるため、`call_frame_pop` を 1 回実行！ (**Frame A が pop される**)。
- 復元されたコンテキストは、元 Caller が SVC 1 を撃った直後の状態！
- 例外復帰 5: 復帰先は元 Caller の SVC 直後 (`armv8m.hpp:418`)。結果が Caller に渡る。

1.3.3 120B context_t 複写の位置

この 1 段の Method Call の間に、PSP スタック上には 3 つのフレームが順次 push される。 各 push において、`call_frame_push` (`dispatch.cpp:117–120`) は以下のコピーを行う:

1. `header->saved = *context`: 120 バイトの `context_t` 全体を構造体代入 (`memcpy`)。
2. `memcpy(callee_frame, context->sp, frame_bytes)`: 32 バイトの例外フレームを複製。

3 回の push (Frame A, Frame B, Frame C) で、**120B コンテキストのメモリコピーが計 3 回 (360B)** 行われ、3 回の pop でそれが逆順に読み戻される。

フレーム	push のタイミング	複写内容と目的
Frame A	SVC 1 (Caller → kobj)	Caller の 120B context_t と 32B HW frame を退避。kobj 実行中の破壊から保護。
Frame B	SVC 2 (kobj → Leaf)	kobj の 120B context_t と 32B HW frame を退避。Leaf 実行中の破壊から保護。
Frame C	SVC 3 (Leaf → kobj EXIT)	Leaf の 120B context_t と 32B HW frame を退避。EXIT_METHOD 実行用の枠。

1.3.4 cost3 実測サイクル数と区間分析

`cost3_report.md` における 150 MHz RP2350 での実機計測結果 (`cost2_trace.log`) は、この構造的ボトルネックを完全に数値化している。

到着区間	合計 cyc	hits	平均 cyc	実コード対応と内容 (`cost3:24–37`)
`mpu_begin`	1448	5	289.6	push_end → dispatch 後半 (3 回) または dispatch → RETURN pop (2 回)。pending-step CAS を含む。MPU store 前。
`push_end`	1297	3	432.3	`dispatch.cpp:74–144`。サイズ検査、header、**120B context 複写**、例外 frame 複写、幾何検算。
`exception_entry`	1029	5	205.8	hardware entry だけでなく直前 Thread handler (validation、ledger scan、request 作成) の仕事を含む。
`context_saved`	656	5	131.2	`armv8m_ctx.S:34–60`。`CTX_SAVE`、`current_context` 取得、整数・FP callee-saved 退避。
`mpu_end`	643	5	128.6	`arch_glue.cpp:52–100`。lookup、profile 判定、Region 2 計算、2/3 store、hook。
`restore_end`	602	5	120.4	`armv8m_ctx.S:77–117`。コンテキスト復元、PSPLIM、CONTROL、ISB。`bx lr` は含まない。
`push_begin`	396	3	132.0	`dispatch.cpp:203–218`。分岐、request 複写・検査。
`dispatch`	328	5	65.6	`armv8m_ctx.S:134`。glue と dispatch 入口。
`handler_begin`	164	2	82.0	hardware unstack 後の handler trampoline prologue/hook。
`return_stub`	141	2	70.5	`armv8m_ctx.S:167–172`。Leaf 処理・return を含む。
`handler_end`	106	1	106.0	primitive RETURN 後の `call_method` 後半。
`caller_resume`	81	1	81.0	最終 restore_end → unstack → Caller 復帰。

重要な実測の結論:

1. **1 境界あたりの基本オーバーヘッド**: `context_saved` (131.2) + `dispatch` (65.6) + `mpu_end` (128.6) + `restore_end` (120.4) = **約 446 traced cyc/境界**。5 回の境界越えだけで $446 \times 5 \approx 2230$ cyc を占める。
2. **push の重さ**: `push_end` は 1 hit あたり **432.3 cyc**。120B のメモリコピーとスタック幾何検算が大きな負荷となっている。
3. **MPU 書込みは主因ではない**: MPU body (MMIO への実際の書込み) は 5 回合計で 80 cyc、すなわち **16 cyc/更新** に過ぎない (`cost3:72`)。MPU 書込みをスキップする実験を行っても、untraced call1 は 5774 → 5830 cyc とむしろ悪化した (`cost3:73`)。

結論として、V2 のボトルネックは「MPU 設定の重さ」ではなく、**「中継による 5 回の境界越え」と「3 回の 120B context 複写」** であった。

—

1.4 V3 における SVC と Method Call の再設計

1.4.1 V3 の基本哲学

V3 設計書 (`docs/design/kernel_v3.typ`) は、V2 の計測結果を踏まえ、ハンドラ構造を根本から再構築する。

- **カーネルとカーネルオブジェクトの統合**: カーネルと kobj を一体のカーネルとして扱い、Thread mode のカーネル処理はその世代のカーネル actor ID を持つ (決定 1–11)。
- **二段構えの全廃**: Caller → kobj → primitive CALL という二段構えを廃止する。
- **静的属性による経路分離**: システムコールの実行モードを、実行時の動的探索ではなく、API ID ごとの静的 traits で決定する。

1.4.2 V3 の同期 SVC: Handler mode 完結経路 (S0)

`GET_CURRENT_OBJECT` や短い stream batch、CAS などの同期システムコールは、Thread mode のカーネルハンドラを起こさず、**Handler mode の同一例外内で完結して即座に呼び出し元へ復帰する** (S0 経路)。

項目	V2 の実装	V3 の設計 (S0)
境界越え回数	2 回 (SVC 2 回、例外復帰 2 回)	**1 回** (SVC 1 回、例外復帰 1 回)
スタック操作	push 1 回、pop 1 回 (120B copy)	**push / pop ゼロ** (スタック変更なし)
レジスタ保存	Caller の全 context 退避 (`r4–r11`, PSP, FP)	**最小限の C ABI 保存のみ** (MSP 上)
`CONTROL` / `PSPLIM`	特権への切替えと復元	**変更なし** (Caller の状態を維持)
MPU 窓	再計算・2 回の更新	**更新なし** (同一保護領域を維持)
復帰方法	kobj が `return_stub` から再 SVC	**Handler mode から直接 `bx lr`**

1.4.2.1 S0 のステップバイステップ推移

1. **Caller (Thread mode)**: `r0=5` (`GET_CURRENT_OBJECT`) で `svc 0` を発行。
   - ハードウェアが PSP に 8 語 (HW Frame) を push。
2. **Handler mode 入口**:
   - `LR` (`EXC_RETURN`) を MSP 上に保持。
   - PSP から HW Frame のアドレスを取得。
   - `r4–r11` をスレッドの `context_t` に丸ごと退避することは **しない** (C 関数の prologue が MSP 上に退避する通常の AAPCS 保存のみ)。
3. **ハンドラ本体の実行**:
   - 静的ディスパッチャが `handler_service<GET_CURRENT_OBJECT>` を直接呼び出す。
   - 保存された origin 情報と特権台帳から Caller の actor ID を読み出し、HW Frame の `r0` (エラー=0) と `r1` (オブジェクト ID) に直接書き込む。
4. **例外復帰**:
   - PSP、`CONTROL`、`PSPLIM`、MPU の書き戻しは行わない (同一条件のため)。
   - 保持していた `EXC_RETURN` で直ちに `bx lr` を実行！
5. **Caller (Thread mode)**:
   - 呼び出し直後の命令へ 1 回の往復で復帰。

[設計目標]: net min 1985 cyc → **500 cyc 以下** (`kernel_v3.typ:600`)。 **[Handler mode の制約]**: Handler mode で完結させるため、最悪実行時間 $H_{\max}$ を 10 µs 以下、割り込み禁止区間を 2 µs 以下に抑える (`kernel_v3.typ:235`)。ブロッキング、yield、動的メモリ確保、再帰的 SVC は厳格に禁止される。

1.4.3 V3 の Method Call: 開始 1 回・終了 1 回 (S1/S2)

V3 における `CALL_METHOD` は、中継アクターを挟まず、**開始 1 回・終了 1 回の計 2 回の境界越え** に統合される。

項目	V2 の実装	V3 の設計 (S1/S2)
境界越え回数	5 回 (SVC 5 回、例外復帰 5 回)	**2 回** (開始 1 回、終了 1 回)
スタックフレーム	push 3 回、pop 3 回 (物理 3 枚)	**logical method frame 1 枚**
120B context 複写	3 回 memcpy (計 360B)	**廃止** (immutable descriptor 参照化)
中継 actor	親 kobj が Thread mode で中継	**全廃** (kernel landing が直接 tail transfer)
復帰プロトコル	Leaf return_stub → EXIT → RETURN	**context reuse 専用復元経路** (`reused_context`)

1.4.3.1 [開始] CALL 開始のステップバイステップ推移

1. **Caller (Thread mode)**: `CALL_METHOD` を SVC 発行 (SVC 1)。
2. **Handler mode 入口 (S1)**:

- Caller の `r4–r11`、PSP、`EXC_RETURN` を Caller の continuation に 1 回保存。
- 静的ディスパッチャが `prepare_kernel_landing` を呼び出す。
- **例外復帰先を特権 Thread mode の kernel landing に設定。**

3. **例外復帰 → Kernel Landing (Thread mode, 特権)**:
   - kernel landing は専用の kernel stack segment で動作。
   - 対象オブジェクト/メソッドの immutable descriptor を pin し、権限・最大深さを検査。
   - 特権台帳に logical method frame を 1 枚 commit。
   - **Tail Transfer Gate**:
     - Callee の stack segment、メソッド引数、`LR = return_stub`、MPU 保護属性を適用。
     - Callee へ直接ジャンプ (tail branch)！

1.4.3.2 [終了] CALL 終了のステップバイステップ推移

1. **Leaf (Thread mode)**: 処理を終えて C return すると、共通 `return_stub` から `EXIT_METHOD` を SVC 発行 (SVC 2)。
2. **Handler mode 入口**:
   - 発行 actor、method frame 世代、depth を特権台帳で検算。
   - **context reuse 専用の復元経路 (`resume_reused_context`)**:
     - 開始時に保持された Caller の continuation から、元 PSP、callee-saved (`r4–r11`)、元 HW frame、`CONTROL`、`PSPLIM`、MPU descriptor を復元。
     - 元 Caller の HW frame の `r0`/`r1` に戻り値を設定。
3. **例外復帰**:
   - Caller の元 HW frame に対して Handler mode から直接例外復帰 (`bx lr`)！
4. **Caller (Thread mode)**:
   - SVC 1 の直後へ復帰。

1.4.3.3 なぜ「普通の C return による例外復帰」を禁止するのか？

設計書で強調される重要なハードウェア制約 (決定 3): **Cortex-M の Thread mode は、単に PSP や特権レジスタを準備しただけでは `EXC_RETURN` を実行できない。** Handler mode 以外の文脈で `bx lr` (LR が `EXC_RETURN`) を実行しても、ハードウェア例外復帰はトリガされず HardFault となる。 したがって、Thread mode のカーネルが普通の C return で例外復帰することは不可能であり、**「context reuse で入った呼出しは、context reuse 専用の復元経路 (Handler mode の復帰ゲートまたは専用トラップ) で戻す」** 規約が不可欠となる (`kernel_v3.typ:264–277`)。

1.4.3.4 120B context_t 複写の廃止

V2 では push/pop ごとに 120B の `context_t` 全体をスタック上に memcpy していたが、V3 では保護領域やスタック境界を **immutable descriptor への参照** とする。これにより、保存が必要なデータは 32B の callee-saved 整数レジスタと数語のメタデータのみに圧縮される (`kernel_v3.typ:321–325`)。

[設計目標]: call1 net min 5774 cyc → **3000 cyc 以下**、call2 11642 cyc → **6000 cyc 以下** (`kernel_v3.typ:601`)。

1.4.4 設計書における確定事項・未確定事項・棚上げ事項の境界

V3 設計書を正確に読むために、何が決定済みで何が未確定かを峻別する。

区分	項目	設計書上の扱い・理由
確定 (決定 1–4, 6–7, 9–11)	Handler mode 完結	GET, atomic, stream 一部は H/S0 完結。IRQ 予算は 10 µs / mask 2 µs。
確定	専用戻り経路	context reuse で入った呼出しは専用の復元経路で戻す (決定 3)。
確定	レジスタ保存契約	現行の保存契約 (r4–r11, flags, scratch) は維持する (決定 4)。
確定	Atomic 範囲	1 ID + op。カーネル H モードは CAS と load/store のみ。FETCH_ADD やキュー等は objectland で処理 (決定 6)。
確定	Wait queue 不採用	専用 wait/wake キューは作らない。mutex は CAS と宛先 YIELD、値待ちは再確認 (決定 9)。
確定	lane の両立	native は静的 traits、objectland は raw SVC の親ハンドラインターセプトを完全維持 (決定 2)。
未確定 (決定 8)	宛先 YIELD の世代	スレッド枠の再利用 (世代) の扱い。上層案は「YIELD はヒント扱い、期待世代は ABI に入れない」だが、決定ではなく未確定として扱う (`kernel_v3.typ:484–488, 677`)。

区分	項目	設計書上の扱い・理由
未確定	Atomic API 番号	番号割当てと引数 layout は `//abi` 正本に合わせて別途確定 (`kernel_v3.typ:459–465`)。
棚上げ (決定 5)	動的ホットスワップ	呼出し完了待ちが他システムへ波及するため棚上げ。actor ID を定数に焼かない制約のみ残す (`kernel_v3.typ:386–388, 674`)。

—

1.5 静的属性表と template dispatcher

1.5.1 6 つの静的属性

V3 では、全システムコールの振る舞いが静的属性構造体 `svc_attribute` によって定義される (`internal_headers/shizuku/svc_attributes.hpp:11–17`):

1. `permission`: `unprivileged_allowed` (非特権可) または `privileged_only` (特権のみ)。
2. `from_isr`: ISR (割り込みハンドラ) から直接呼出し可能か。
3. `mode`: `handler` (Handler mode 内完結) または `thread` (Thread mode の kernel actor)。
4. `save_level`: `minimal` (S0: 最小入口) または `continuation` (S1/S2: コンテキスト保存)。
5. `authority`: `none`、`create_flags`、`parent_ledger` などの特殊権限述語。
6. `work_bound`: 実行時間上限 ($H_{\max}$)。

現行 31 個の API の分類 (`svc_attributes.hpp:20–36`, `kernel_v3.typ:102–136`):

- **Handler mode 完結 (H/S0)**: `GET_CURRENT_OBJECT` (5), `GET_CALLER_OBJECT` (6), `MEMORY_OWNER` (16), `OBJECT_NAME` (18), `STREAM_OPEN` (20), `STREAM_PUSH_N` (29), `STREAM_POP_N` (30), `STREAM_AVAILABLE` (31), および `ATOMIC` (CAS/load/store)。
- **ISR から呼出し可能 (`from_isr=true`)**: 18, 20, 29, 30, 31, および `ATOMIC`。
- **特権限定 (`privileged_only`)**: `GRANT_REGION` (23), `KILL_THREAD` (25)。

1.5.2 template dispatcher の仕組み

ディスパッチャは、C++ の `if constexpr` を用いたテンプレート関数として生成される (`kernel_v3.typ:158–184`):

```
template<Api Id, class Arch>
DispatchResult invoke(Origin const& origin, EntryView& entry) {
  using A = svc_traits<Id, Arch>;
  if constexpr (A::permission == privileged_only)
    if (!origin.is_privileged()) return reject(NOT_PRIVILEGED);
  if (origin.from_isr() && !A::from_isr) return reject_context();
  if (!A::permission_predicate(origin, entry.args)) return reject_permission();
  if constexpr (A::mode == handler) {
    static_assert(A::bounded && !A::may_yield && !A::may_block);
    return handler_service<Id>(origin, entry); // 同じ例外内で完結
  } else {
    return prepare_kernel_landing<Id, A::save_level>(origin, entry); // Thread landing へ
  }
}
```

この設計により、実行時の関数ポインタ配列走査や権限の動的再判定が完全に排除され、コンパイル時に最短の命令列が展開される。

1.5.3 native lane と objectland lane の分離

V3 における極めて重要な設計判断は、**native レーンと objectland レーンの分離** である (決定 2)。

項目	native lane	objectland lane
所属ドメイン	`svc_domain::native`	`svc_domain::objectland`
確定契機	生成時 (コンポジション時) に固定	生成時 (コンポジション時) に固定
ディスパッチ方式	上記の `svc_traits` による template 分岐	固定親ハンドラ (`parent_handler`) への一律 Thread ディスパッチ
raw 番号の扱い	未知の番号は即座に `UNKNOWN_API` で拒否	raw 番号を解釈せず、親ハンドラへそのまま透過
親の独自意味論	なし (explicit method を推奨)	親による番号の独自解釈、`FORWARD_CHILD_EXIT` の親 ledger 検査を完全維持

注意すべき原則: native の高速ディスパッチ表を objectland の raw SVC へ適用してはならない (`kernel_v3.typ:30, 349–371`)。 objectland では、親オブジェクトが子の raw SVC をインターセプトして独自のエミュレーションやポリ

モーフィズムを行う意味論 (`source/selftest/handler_binding.cpp` の親 A/B による同一番号の独自解釈など) が存在しており、これを native 表で先に弾いてしまうと既存の互換性が崩壊するためである。

—

1.6 まとめと読み解きの注意点

1.6.1 境界越え回数と性能目標の総括

処理内容	V2 境界回数	V3 目標回数	V2 実測 cyc (net min)	V3 目標 cyc (net min)
同期 SVC (GET 等)	2 回	**1 回**	1985 cyc	**500 cyc 以下**
Method call 1 段	5 回	**2 回**	5774 cyc	**3000 cyc 以下**
Method call 2 段	10 回	**4 回**	11642 cyc	**6000 cyc 以下**
共通境界コスト	約 446 cyc/境界	**200–250 cyc**	実測 (cost3)	部品目標
フレーム準備	約 432 cyc/frame	**100–150 cyc**	実測 (cost3)	部品目標

1.6.2 作者本人が設計書を読み解く上での 5 つの急所

1. **「Handler mode 完結」の境界条件を誤解しないこと**:
   - H 完結は、Thread mode の仕事を Handler に無理やり詰め込むことではない。有界 ($H_{\max} \leq 10$ µs)、非ブロック、非割り込み禁止 ($\mathrm{PRIMASK} \leq 2$ µs)、メモリコピー上限が証明できる操作に限定される。
2. **C return を例外復帰と見なす実装は絶対に作らないこと**:
   - Cortex-M のハードウェア特性上、Thread mode からの `bx lr` は EXC_RETURN を解釈しない。context reuse 専用の復帰ゲートが不可欠である。
3. **レジスタ保存契約の緩和を安易な最適化と過信しないこと**:
   - r4–r11 の保存契約や flags の保存を削っても、H 経路ではハードウェアが既に保全しているため 0–4 cyc (0–0.8%) 程度しか変わらない (`kernel_v3.typ:560`)。契約を崩して安全性を犠牲にする利益はない。
4. **native と objectland を混同しないこと**:
   - native の最適化 (template dispatch) を急ぐあまり、objectland の親ハンドラ機構を破壊してはならない。生成時固定の `svc_domain` で明確に分離する。
5. **確定事実と設計予算・見積もりを混同しないこと**:
   - cost3 の数値 (1985 cyc, 5774 cyc, 446 cyc/境界) は 150 MHz RP2350 で採取された **[実測]** である。
   - 一方、V3 の 500 cyc や 3000 cyc は **[設計目標 (予算)]** であり、実装と実機検証を経て初めて確定する。